UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Experiment No. 8

Student Name: Sanchit Katoch
UID: 25MCA20059
Branch: MCA(GEN)
Section/Group: 25MCA-1_A
Semester: 2nd
Date of Performance: 31/03/26
Subject Name: Technical Training-1
Subject Code: 25CAP-652

Aim:

Apply stored procedure concepts for database operations

Objective:

- To understand the concept of stored procedures in PostgreSQL.
- To learn how to pass parameters using IN, OUT, and INOUT modes.
- To perform database operations like updating records using procedures.
- To implement error handling using exception blocks in PL/pgSQL.

Tools Used:

PostgreSQL

Procedure:

Step1: Create the employees4 table and insert sample data.
Step2: Define the stored procedure update_salary using PL/pgSQL.
Step3: Retrieve the employee's current salary and validate existence.
Step4: Update the salary by adding the given increment.
Step5: Handle errors using the EXCEPTION block.
Step6: Execute the procedure using CALL and verify results with SELECT.

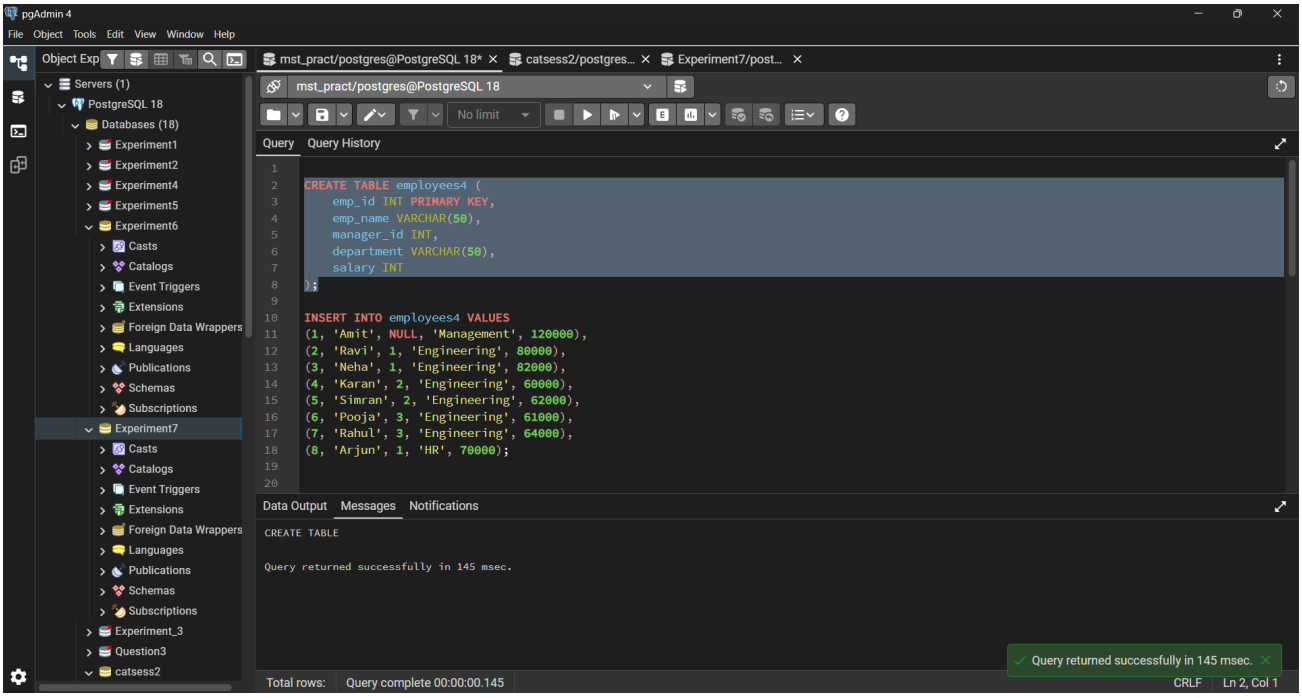
Code:

```
CREATE TABLE employees4 (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   manager_id INT,
   department VARCHAR(50),
   salary INT
);

INSERT INTO employees4 VALUES
(1, 'Amit', NULL, 'Management', 120000),
(2, 'Ravi', 1, 'Engineering', 80000),
(3, 'Neha', 1, 'Engineering', 82000),
(4, 'Karan', 2, 'Engineering', 60000),
(5, 'Simran', 2, 'Engineering', 62000),
(6, 'Pooja', 3, 'Engineering', 61000),
(7, 'Rahul', 3, 'Engineering', 64000),
(8, 'Arjun', 1, 'HR', 70000);

CREATE OR REPLACE PROCEDURE update_salary(
   IN p_emp_id INT,
   INOUT p_salary NUMERIC(20,3),
   OUT status VARCHAR(30)
)
AS
$$
DECLARE
   curr_sal NUMERIC(20,3);
BEGIN

   SELECT salary INTO curr_sal
   FROM employees4
   WHERE emp_id = p_emp_id;

   IF NOT FOUND THEN
      RAISE EXCEPTION 'EMPLOYEE NOT FOUND';
   END IF;

   curr_sal := curr_sal + p_salary;
```

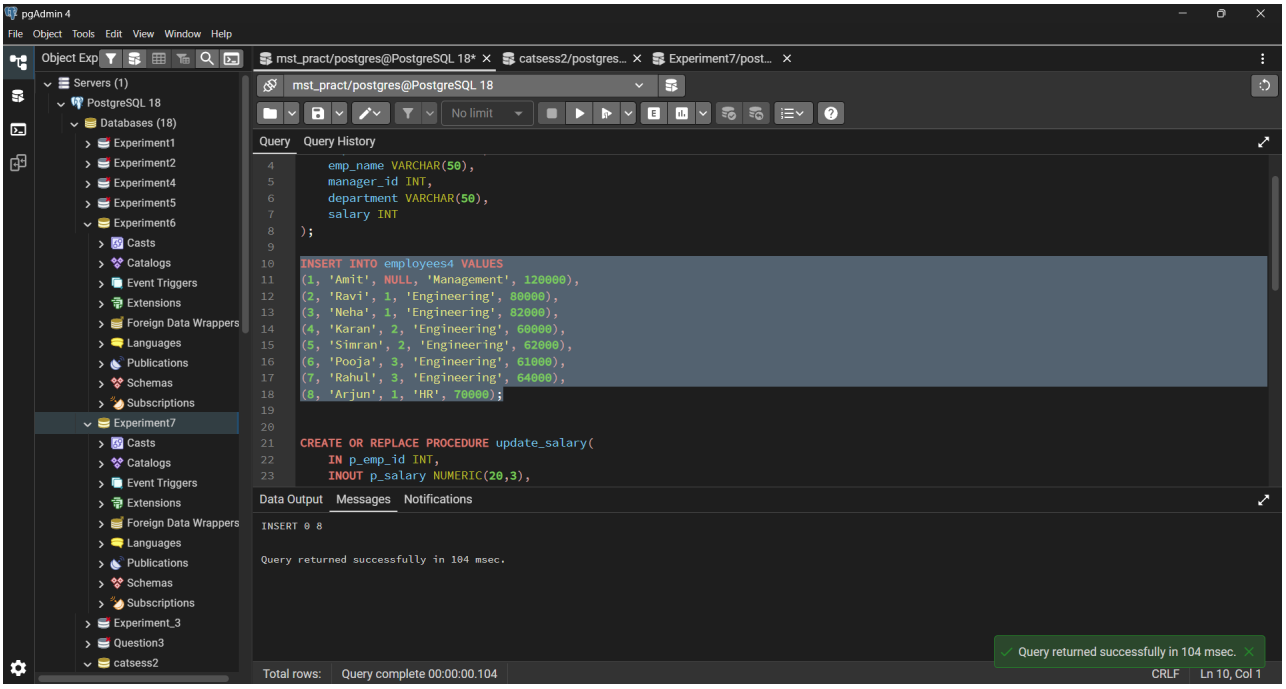
```
    UPDATE employees4
    SET salary = curr_sal
    WHERE emp_id = p_emp_id;

    p_salary := curr_sal;
    status := 'SUCCESS';

EXCEPTION
    WHEN OTHERS THEN
        status := 'FAILED';
END;
$$ LANGUAGE plpgsql;

SELECT * FROM employees4
```

Output:

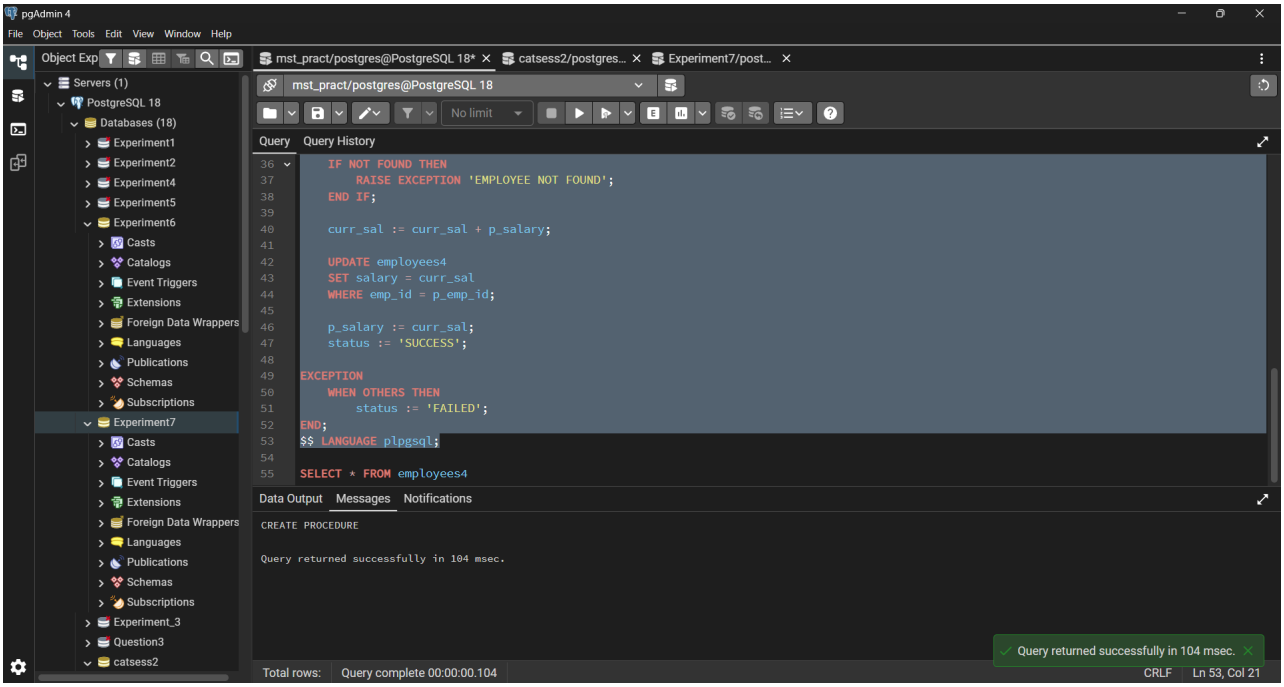
Create table



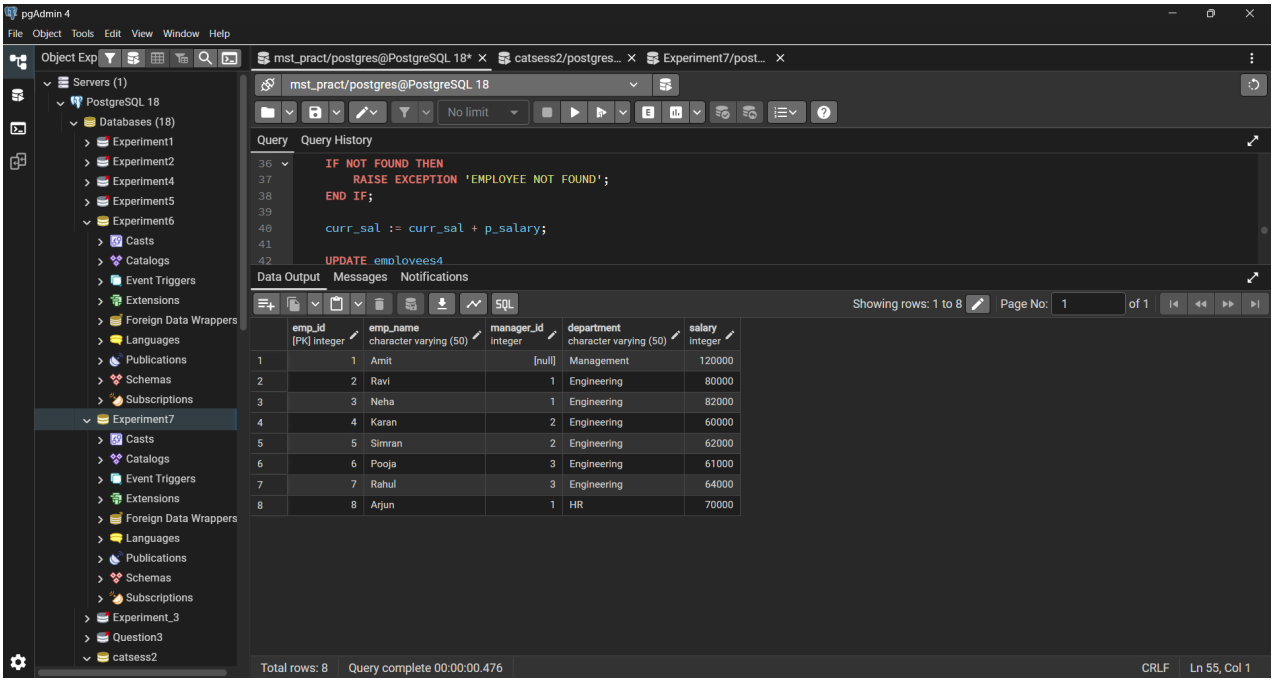
Insert data



Create Procedure



Select Query



Learning Outcomes:

- Understood how to create and use stored procedures in PostgreSQL.
- Learnt the use of IN, OUT, and INOUT parameters in procedures.
- Gained knowledge of updating database records using procedures.
- Understood how to handle exceptions using EXCEPTION block.
- Learnt how to retrieve and modify data dynamically inside procedures.
- Practiced real-world database operations like salary updates.